

Simple, efficient, sound-and-complete combinator parsing for all context-free grammars, using an oracle

Tom Ridge

University of Leicester

Abstract

Parsers for context-free grammars can be implemented directly and naturally in a functional style known as “combinator parsing”, using recursion following the structure of the grammar rules. Traditionally parser combinators have struggled to handle all features of context-free grammars, such as left recursion.

In previous work, Ridge defined novel parser combinators that allowed to parse all context-free grammars. He proved, in the HOL4 theorem prover, that a parser generator built using these combinators was both *sound* and *complete*. Unfortunately the performance was not as good as other parsing methods such as Earley parsing.

In this paper, we build on this previous work, and combine it in novel ways with existing parsing techniques such as Earley parsing. The result is a combinator parsing library that *can handle all context-free grammars, and has good performance*.

Categories and Subject Descriptors D.1.1 [Applicative (Functional) Programming]; F.4.2 [Grammars and Other Rewriting Systems]; Parsing; I.2.7 [Natural Language Processing]: Parsing

General Terms Parsing, Functional programming, Combinator parsing, Earley parsing, Context-free grammar

Keywords parsing, functional, combinator, Earley, context-free

1. Introduction

In previous work [15] Ridge introduced novel parser combinators that were capable of parsing all context-free grammars. For example, a parser for the grammar $E \rightarrow E E E \mid "1" \mid \epsilon$ can be written in OCaml as:

```
let rec parse_E = (fun i -> mkparser "E" (
  (parse_E **> parse_E **> parse_E)
  ||| (a "1")
  ||| eps) i)
```

In [4] Barthwal and Norrish discuss this work:

[Ridge] presents a verified parser for all possible context-free grammars, using an admirably simple algorithm. The drawback is that, as presented, the algorithm is of complexity $O(n^5)$.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

POPL '14 2014-01-22, San Diego

Copyright © 2014 ACM [to be supplied]...\$15.00

Existing techniques such as Earley parsing [5] take time $O(n^3)$ in the length of the input in the worst case. Thus, Ridge’s algorithm is certainly not optimal in this regard. However a more subtle problem, arguably more important, is also present. Experience shows that the previous algorithm *tends to achieve its worst case run time on many classes of grammar*. This is in marked contrast to Earley parsing, which although $O(n^3)$ in the worst case, usually performs very well in practice. For example, common classes of grammar can be parsed in linear time using Earley’s algorithm [13].

In this work, we seek to address these performance problems. We have three main goals for our parsing library.

- The library should provide an interface based on parser combinators.
- The library should handle all context-free grammars.
- The library should have “good” performance.

Our chosen interface is parser combinators, and as with the previous work, we desire to parse arbitrary context-free grammars (we motivate these choices in Section 2). The challenge is to achieve these goals whilst providing Earley-like performance: $O(n^3)$ in the worst case, but typically much better on common classes of grammar. Our main contribution is to show how to combine a combinator parsing interface with an efficient general parsing algorithm such as Earley parsing. Another contribution is the concrete implementation of this approach in the P3 combinator parsing library. We now briefly outline our new approach, and then give an overview of the rest of the paper.

Consider the problem of parsing an input string s , given a grammar Γ (a finite set of rules) and a nonterminal start symbol S . In general, we will work with substrings $s_{i,j}$ of the input s between a low index i and a high index j , where $i \leq j$. In symbols we might write the parsing problem as $\Gamma \vdash S \rightarrow^* s_{i,j}$. Suppose the grammar contains the rule $S \rightarrow AB$. Then one way to derive $\Gamma \vdash S \rightarrow^* s_{i,j}$ is to derive $\Gamma \vdash A \rightarrow^* s_{i,k}$ and $\Gamma \vdash B \rightarrow^* s_{k,j}$:

$$\frac{\Gamma \vdash A \rightarrow^* s_{i,k} \quad \Gamma \vdash B \rightarrow^* s_{k,j}}{\Gamma \vdash S \rightarrow^* s_{i,j}} (S \rightarrow AB) \in \Gamma$$

This rule resembles the well-known *Cut* rule of logic, in that it introduces an unknown k in the search for a derivation. The problem is that there is no immediate way to determine the possible values of k when working from the conclusion of the rule to the premises. Put another way, a top-down parse of the substring $s_{i,j}$ must divide the substring into two substrings $s_{i,k}$ and $s_{k,j}$, but there is no information available to determine the possible values of k . Attempting to parse for all k such that $i \leq k \leq j$ results in poor real-world performance.

The traditional solution provided by combinator parsing is as follows. Rather than seek solutions for the problem $\Gamma \vdash S \rightarrow^* s_{i,j}$

²Footnotes describe how mini-P3 differs from P3.

tation language is a small subset of OCaml, essentially the simply typed lambda calculus with integers, strings, recursive functions, records and datatypes³. Apart from memoization, the code is purely functional. It should be very easy to re-implement our approach in other functional languages such as Haskell, Scheme and F#. The full code for mini-P3 and P3 can be found in the online resources <http://www.tom-ridge.com/p3.html>.

The structure of the rest of the paper is as follows. In Section 2 we motivate our design goals. In Section 3 we discuss preliminaries and in Section 4 we introduce the basic types such as those for substrings and grammars. The subsequent sections modularly introduce different aspects of our approach. The fact that different aspects are handled modularly is a strength of our approach. We start by discussing the types related to parsers in Section 5, and then define the sequencing and alternative combinators in Section 6. In Section 7 we introduce our running example, which we develop further in Sections 9, 11 and 13. Initially we deal with grammars which do not contain the terminal ϵ , and hand code the oracle. As we progress further we extend our example so that we end up with efficient, memoized parsers capable of dealing with all context-free grammars. In Section 8 we describe the Earley parsing phase and the construction of the oracle. In Section 10 we recall the notion of parsing context and show how to combine this with our parsing combinators. In Section 12 we implement memoization to make the action phase efficient. In Section 14 we report on various experiments to measure performance. In Section 15 we discuss related work, and in Section 16 we conclude.

2. Motivation for design decisions

In the previous section, we listed several goals for our parsing library. In this section, we briefly justify the desire to handle all context-free grammars, and for an interface based on parser combinators.

Why all context-free grammars? The class of all context-free grammars includes many classes of grammar that are not handled well (or at all) by existing parsing tools. For example, one such class is that of ambiguous grammars. Ambiguous grammars are essential when parsing (ambiguous) natural languages, but in computer science it is less clear that there is a need to handle a wide class of grammars. Parsing techniques for restricted classes of grammar will tend to perform better than more general approaches, and real-world performance is often the main factor when choosing a parsing technique. On the other hand, restricted parsing techniques introduce complexity: at the very least, the user must be able to decide whether the particular grammar they are interested in can be handled by a particular tool. This can be difficult, because the descriptions of the grammars that a given tool can handle are often complicated even at the theoretical level, and may refer to implementation details of the tool itself. If the grammar is not accepted by a given tool, it is sometimes possible to alter the grammar so that it is accepted. However, this is undesirable for several reasons, for example, it can make the grammar much more difficult to modify and maintain. Absent a clear description of the class that a particular tool can handle, one can attempt to use the tool itself. A given tool may reject a grammar outright (with an error message that may, or may not, be comprehensible), or it may accept the grammar, but

fail on particular inputs. A parsing failure is often accompanied by an error message, such as “shift/reduce conflict”, which is incomprehensible without some understanding of the underlying parsing machinery. Our main motivation for handling all context-free grammars is that this choice makes our library *conceptually simple*: the user never encounters the sorts of difficulties described above with restricted parsers. The price of supporting this generality is that we should expect that the performance will not be competitive with more specialized techniques.

Why combinator parsing? Traditional tools typically require grammars to be defined in a “grammar file”, with a specialized syntax. These files are then compiled by a parser generator to produce the actual parser code. One advantage of this approach is that the syntax of the grammar file can be very clean, allowing the reader to focus clearly on the details of the grammar itself. Some disadvantages are that the programmer must learn a new syntax for grammar files, the syntax supports only a limited number of pre-defined constructs (such as iteration), there are new tools to learn such as the parser generator, there are usually at least two additional files involved (the grammar file, and the generated parser), and the generated parser code is not easily editable by the programmer.

The structure of combinator parsers also corresponds closely to the original grammar, although the result is typically less clean than when using a specialized syntax. With parser combinators, the programmer does not have to learn a new syntax for grammar files, nor the details of a new tool to compile such files. Functional programmers find parser combinators appealing because they are simply higher-order functions in the host language. As such, they can be parameterized (perhaps by other parsers), and new combinators can be defined for particular application domains. The ability to manipulate parsers as first-class functions contributes greatly to the flexibility of this approach. Different parsers can be combined in a modular manner. The integration with the host language is also smooth. For example, when writing parsers, the full capabilities of the host language are available (including the type system and modules), with all the attendant benefits.

Combinator parsers, and other recursive descent parsers, execute in a top-down manner, where the control flow follows the structure of the grammar. In particular, the simplicity of the sequencing of the various computations makes it easy to understand the behaviour of such parsers when they are combined with effects, for example, in the “parsing actions”. Combinator parsers are usually easy to debug, again because the control flow is straightforward.

3. Mathematical preliminaries

In this section we review some background material, using informal mathematics. In the following sections, where necessary, we clarify the connection between these definitions and the formal code.

Substrings $s_{i,j}$ are triples of a string and two integers, representing the part of the string s between indexes i and j . If the length of s is n , then $0 \leq i \leq j \leq n$. Two substrings $s_{i,k}$ and $s_{k,j}$ can be concatenated to give the substring $s_{i,j}$.

Symbols are the disjoint union of the set of nonterminals and the set of terminals. Nonterminals are written e.g. E , terminals e.g. “1”. The terminal corresponding to the empty string is traditionally written ϵ . To each terminal is associated a set of substrings corresponding to that terminal. For example, the terminal “1” is associated to the set of substrings of the form $s_{i,i+1}$, where the character at index i in s is the character 1. The terminal ϵ is associated to the set of substrings of the form $s_{i,i}$. A rule is a pair of a nonterminal and a list of symbols, written e.g. $S \rightarrow A B$ or $E \rightarrow \text{“1” } E \text{ “1”}$. The right-hand side of a rule cannot be the empty list. The variables α, β are typically used to represent finite sequences (lists) of symbols. The bar notation is shorthand for different alternatives e.g. E

³In the following sections we have lightly typeset the OCaml code. The sequencing combinator `***>` is written $\otimes$ and associates to the right; the alternative combinator `|||` is written $\oplus$; and the action function `>>>` is written $\gg$. The notation $s.[i]$ denotes the i th character of the string s . Records with named fields are written e.g. $\langle f1 = v1; f2 = v2 \rangle$. Functional record update is written $\langle r \text{ with } f1 = v1; f2 = v2 \rangle$. Otherwise the OCaml syntax we use should be readily understandable by anyone familiar with functional programming.

```

type ( $\alpha, \beta$ ) fmap = ( $\alpha \times \beta$ ) list

type substring = SS of string  $\times$  int  $\times$  int

type term = string

type nonterm = string

type symbol = NT of nonterm | TM of term

type rhs = Atom of symbol
          | Seq of symbol  $\times$  symbol
          | Alt of symbol  $\times$  symbol

type parse_rule = nonterm  $\times$  rhs

type grammar = parse_rule list

type raw_parser = substring  $\rightarrow$  substring list

type ty_oracle = (symbol  $\times$  symbol)  $\rightarrow$  substring  $\rightarrow$  int list

type local_context = LC of (nonterm  $\times$  substring) list

let empty_fmap = []
let empty_oracle = (fun (sym1, sym2)  $\rightarrow$  fun ss  $\rightarrow$  [])
let empty_context = (LC [])

```

Figure 1. Basic types and trivial values

$\rightarrow E E E \mid "1" \mid \epsilon$ denotes the three rules $E \rightarrow E E E$, $E \rightarrow "1"$ and $E \rightarrow \epsilon$. A grammar is a finite set of rules.

Parse trees are finitely branching trees consisting of nodes (each decorated with a nonterminal) and leaves (each decorated with a terminal and a substring representing the part of the input that was parsed by that terminal). The substring at a leaf should be in the set associated to the terminal at that leaf. Given a grammar Γ , a node in a tree decorated by a nonterminal X , and children of that node decorated by symbols $S_0, \dots, S_m$, there should be a rule $X \rightarrow S_0 \dots S_m$ in Γ . Moreover, the substrings at the leaves should match up in the obvious way: if two adjacent leaves have substrings $s_{i,j}$ and $s'_{j',k}$ then $s' = s$ and $j' = j$.

Given a grammar Γ , parse trees give rise to a relation between symbols X and the substrings $s_{i,j}$ that “can be parsed as” an X . In symbols, this relation is written $\Gamma \vdash X \rightarrow^* s_{i,j}$. The substring $s_{i,j}$ can be parsed as the symbol X if there is a parse tree with X at the root, such that the concatenation of the substrings at the leaves gives $s_{i,j}$. Similarly, a substring $s_{i,j}$ can be parsed as the sequence $X Y$ if there exists k such that $s_{i,k}$ can be parsed as an X , and $s_{k,j}$ can be parsed as a Y . This definition is extended inductively to finite sequences of symbols in the obvious way.

Given a grammar Γ , a start symbol S , and an input string s of length n , parsing involves determining the parse trees rooted at S such that the concatenation of the substrings at the leaves gives $s_{0,n}$.

4. Basic types

In Fig. 1 we give types for finite maps (represented by association lists), substrings, terminals, nonterminals, symbols, the right-hand sides of parse rules, parse rules, and grammars. Alternatives occurring as right-hand sides of rules are not treated as shorthand, but are captured explicitly. Note that the `rhs` type permits only unary rules (e.g. $E \rightarrow F$) and binary rules (e.g. sequences $E \rightarrow A B$ or

```

type ( $\alpha, \beta, \gamma$ ) sum3 = Inl of  $\alpha \mid$  Inm of  $\beta \mid$  Inr of  $\gamma$ 

type inl = unit
type outl = symbol

type mid = <
  rules7 : parse_rule list;
  tmparsers7 : (term, raw_parser) fmap >
type inm = mid
type outm = mid

type inr = <
  ss4 : substring;
  lc4 : local_context;
  oracle4 : ty_oracle >
type  $\alpha$  outr =  $\alpha$  list

type input = (inl, inm, inr) sum3
type  $\alpha$  output = (outl, outm,  $\alpha$  outr) sum3

type  $\alpha$  parser3 = (input  $\rightarrow$   $\alpha$  output)

let empty_mid = <rules7 = []; tmparsers7 = empty_fmap>

```

Figure 2. Parser types and trivial values

alternatives $E \rightarrow A \mid B$). This is a restriction on the *internal representation* of the rules and not on the user of the library.

Raw parsers capture the set of substrings associated to a given terminal. They can be more-or-less arbitrary OCaml code⁴. Given a substring $SS(s, i, j)$, a raw parser returns a list of substrings $SS(s, i, k)$ indicating that the prefix $SS(s, i, k)$ could be parsed as the corresponding terminal. For example, the raw parser `raw_a1` consumes a single 1 character from the input:

```

let raw_a1 (SS(s, i, j)) = (
  if i < j && s.[i] = '1' then
    [SS(s, i, i + 1)]
  else
    [])

```

The oracle type captures the idea that an oracle takes two symbols `sym1`, `sym2`, and a substring $SS(s, i, j)$, and returns those integers k such that $SS(s, i, k)$ can be parsed as `sym1`, and $SS(s, k, j)$ can be parsed as `sym2`.

Finally, the type `local_context` is discussed in the section on parsing with context, Section 10.

5. Parser types

In this section we discuss the types related to parsers given in Fig. 2. In our approach, a parser should be viewed as a collection of three separate functions. We first discuss the `sum3` type, and the function `sum3` which converts three separate functions to a single function, and the function `unsum3` which converts a single function of the appropriate form to three separate functions. Following this, we discuss the particular instances of the `sum3` type that we use for our parsers.

The `sum3` type The `sum3` type generalizes the familiar binary sum to three components. Given three functions of type $\alpha \rightarrow \delta$, $\beta \rightarrow \epsilon$ and $\gamma \rightarrow \zeta$, we can form a composite function of type

⁴A raw parser should behave as a pure function, and should return prefixes of its argument. For a fully formal treatment of the parsers associated with terminals see [15].

$(\alpha, \beta, \gamma) \text{ sum3} \rightarrow (\delta, \epsilon, \zeta) \text{ sum3}$. We can define this composite function explicitly:

```
let sum3 (f, g, h) = (fun i → match i with
| Inl l → Inl(f l)
| Inm m → Inm(g m)
| Inr r → Inr(h r))
```

Moreover, this function is invertible, giving rise to a bijection between functions and triples of functions:

```
let unsum3 u = (
let f = (fun x → dest_inl (u (Inl x))) in
let g = (fun x → dest_inm (u (Inm x))) in
let h = (fun x → dest_inr (u (Inr x))) in
(f, g, h))
```

Here we make use of destructors such as *dest_inl*, where *dest_inl* (Inl *x*) = *x*.

We use the functions *sum3* and *unsum3* extensively when defining the parser combinators. In particular, as a function from inputs to outputs, a parser satisfies the extra conditions (not explicit in the type): given an argument of the form Inl *x*, the parser produces a result of the form Inl *x'*, and similarly for Inm and Inr. Parsers *p* of type $\text{input} \rightarrow \alpha \text{ output}$ should be thought of as the sum of three functions, i.e. $p = \text{sum3}(f, g, h)$. Here *f* is the left-component of the parser, of type $\text{inl} \rightarrow \text{outl}$, *g* is the middle-component of the parser, of type $\text{inm} \rightarrow \text{outm}$ and *h* is the right-component of the parser, of type $\text{inr} \rightarrow \alpha \text{ outr}$.

Left component, extracting a symbol from a parser The left component of a parser consists of a function of type $\text{inl} \rightarrow \text{outl}$, that is, from unit to symbol. The idea is that, if *parse_E* is a parser for the nonterminal E, then the expression *parse_E* (Inl ()) should evaluate to Inl (NT "E"). We define the following auxiliary function:

```
let sym_of_parser p = (dest_inl (p (Inl ())))
```

Middle component, extracting a grammar from a parser The middle component of a parser consists of a function of type $\text{inm} \rightarrow \text{outm}$, where *inm* and *outm* are both equal to type *mid*. The middle component of a parser is therefore of type $\text{mid} \rightarrow \text{mid}$. Intuitively the *mid* type represents the grammar associated with a parser. The middle component of a parser such as *parse_E* can be seen as grammar transformer, that takes a grammar and extends it with extra rules.

The type *mid* is a record type with two fields. The first is a list of parse rules. The second is a finite map from terminals to raw parsers. We use the middle component to extract the grammar associated to a parser such as *parse_E*. The idea is that if *parse_E* is a parser for the nonterminal E, then the expression *parse_E* (Inm *m*) should evaluate to a value of the form Inm *m'*, where *m'* is *m* augmented with rules for the nonterminal E (and all nonterminals reachable from E), and the terminal parsers involved in the definition of *parse_E* (and all terminal parsers involved in the definition of nonterminals reachable from E). The function *grammar_of_parser* introduced in Section 1 can then be defined as follows:

```
let grammar_of_parser p = (dest_inm (p (Inm empty_mid)))
```

Right component, recursive descent parser The right component is a function of type $\text{inr} \rightarrow \alpha \text{ outr}$, where $\alpha \text{ outr} = \alpha \text{ list}$. The left and middle components allow a certain amount of introspection on the parsing code that forms the body of parsers such as *parse_E*. The right component is more familiar, and resembles the traditional type of a combinator parser: a function from a string to a list of possible values. We work with substrings rather than strings, so an input *i* of type *inr* contains a component *i.ss4* of type *substring*.

Two additional fields are present: *i.oracle4* is an oracle that indicates how to split the input when parsing a sequence of symbols, and *i.lc4* is a parsing context that allows combinator parsers to handle all context-free grammars. We discuss these additional fields further in the following sections. The output type $\alpha \text{ outr}$ is simply a list of values at an arbitrary type α .

6. Parsing combinators

In the previous section we discussed the $\alpha \text{ parser3}$ type and related types. In this section we give the definition of the sequencing combinator $p1 \otimes p2$. The definition of the alternative combinator $p1 \oplus p2$ follows the sequencing combinator *mutatis mutandis*. The following section illustrates the use of these combinators on a simple example.

Consider the left component of the sequencing combinator. This should take two parsers *p1* and *p2* and produce the left component (a function from unit to symbol) of the parser $p1 \otimes p2$. The definition is:

```
let seq_l p1 p2 = (fun () →
let (f1, -, -) = unsum3 p1 in
let (f2, -, -) = unsum3 p2 in
let rhs = Seq(f1 (), f2 ()) in
mk_symbol rhs)
```

The left component is a function from unit argument () to a symbol representing the sequential combination of the two underlying parsers. We use the auxiliary function *mk_symbol* to generate new symbols for possible right hand sides. These new symbols are always nonterminals. The requirement on *mk_symbol* is simply that it should be injective on its argument: if *mk_symbol rhs'* = *mk_symbol rhs* then *rhs'* = *rhs*⁵. By way of example, with the current implementation, evaluating *mk_symbol* (Seq(NT "E", NT "E")) returns (NT "(E*E)")⁶.

The middle component for the combination $p1 \otimes p2$, of type $\text{mid} \rightarrow \text{mid}$, transforms a list of rules by adding a new rule representing the sequencing of *p1* and *p2*. It should also call the underlying parsers so that they in turn add their rules.

```
let seqm p1 p2 = (fun m →
let NT nt = seq_l p1 p2 () in
if List.mem nt (List.map fst m.rules7) then m
else (
let (f1, g1, -) = unsum3 p1 in
let (f2, g2, -) = unsum3 p2 in
let new_rule = (nt, Seq(f1 (), f2 ())) in
let m1 = { m with rules7 = (new_rule :: m.rules7) } in
let m2 = g1 m1 in
let m3 = g2 m2 in
m3))
```

⁵Related to this is the requirement that users do not annotate two *different* parsers with the *same* nonterminal; the following must be avoided:

```
let rec parse_E = (fun i → mkntparser "E" ... i)
and parse_F = (fun i → mkntparser "E" ... i)
```

There seems no way to enforce this constraint using types. It should be possible to check this constraint dynamically when parsers are evaluated. An alternative is to use a *gensym*-like technique to construct arguments to *mkntparser* automatically.

⁶Generated names should not clash with user names. The traditional solution is to incorporate a "forbidden" character, not available to users, into generated names. A better approach would use a more structured datatype than strings for the names of nonterminals. For simplicity, we stick with strings and assume the user does not use symbols such as * in the names of nonterminals.

Note that the code first checks whether the nonterminal nt corresponding to $p1 \otimes p2$ is already present in the rules. If so, this nonterminal has already been processed, and there is no need to continue further. This check also prevents non-termination of *seqm* when dealing with recursive grammars. If the nonterminal is not present, then the new rule is constructed, added to the list of rules, and then the middle components $g1$ and $g2$ of the parsers $p1$ and $p2$ are invoked in turn, to add their rules.

The right component of the sequencing combinator should take two parsers $p1$ of type α parser3, and $p2$ of type β parser3, and produce the right component of the parser $p1 \otimes p2$, of type $\text{inr} \rightarrow (\alpha \times \beta) \text{ outr}$.

```
let seqr p1 p2 = (fun i0 →
  let sym1 = sym_of_parser p1 in
  let sym2 = sym_of_parser p2 in
  let ks = i0.oracle4 (sym1, sym2) i0.ss4 in
  let SS(s, i, j) = i0.ss4 in
  let fl k = (
    let rs1 = dest_inr (p1 (Inr (i0 with ss4 = (SS(s, i, k))))) in
    let rs2 = dest_inr (p2 (Inr (i0 with ss4 = (SS(s, k, j))))) in
    list_product rs1 rs2)
  in
  List.concat (List.map fl ks))
```

The function works by first determining the symbols $sym1$ and $sym2$ corresponding to the two underlying parsers. It then calls the oracle with the appropriate symbols and substring $i0.ss4 = SS(s, i, j)$. The resulting values for k are bound to the variable ks . For each of these values k , parser $p1$ is called on the substring $SS(s, i, k)$ and $p2$ is called on the substring $SS(s, k, j)$. The results are combined using the library functions *list_product* and *List.concat*. The function *list_product* takes two lists and forms a list of pairs.

The corresponding right component *altr* for the alternative combinator is much simpler: as with traditional combinator parsers, the results of the parsers $p1$ and $p2$ are simply appended.

```
let altr p1 p2 = (fun i →
  let rs1 = dest_inr (p1 (Inr i)) in
  let rs2 = dest_inr (p2 (Inr i)) in
  List.append rs1 rs2)
```

We are now in a position to define the sequential combination $p1 \otimes p2$. This uses the previously defined functions *seq1*, *seqm*, *seqr* to construct a new parser of type $(\alpha \times \beta)$ parser3 from a parser $p1$ of type α parser3 and a parser $p2$ of type β parser3.

```
let p1 ⊗ p2 = (fun i0 →
  let f = seq1 p1 p2 in
  let g = seqm p1 p2 in
  let h = seqr p1 p2 in
  sum3 (f, g, h) i0)
```

The alternative combination $p1 \oplus p2$ is identical, except that *seq1* becomes *alt1* and so on. We also define the “semantic action” function, which takes a parser p of type α parser3 and a function f from α to β and returns a parser of type β parser3, by mapping the function f over the list of values in the right component. Apart from the fact that we now have three components to deal with, this is the approach taken by traditional parser combinators.

```
let p >> f = (fun i → match i with
| Inl _ → (Inl (dest_inl (p i)))
| Inm _ → (Inm (dest_inm (p i)))
| Inr _ → (Inr (List.map f (dest_inr (p i)))))
```

Finally, we turn to the auxiliary function *mkntparser*. This function allows the user to introduce concrete *names* for nontermi-

nals, to label the corresponding code for parsers: let *parse_E* = (fun $i \rightarrow \text{mkntparser}$ "E" ... i). At this stage, we introduce a version of *mkntparser* that does not deal with context. In Section 10 we add the ability to handle context.

```
let mkntparser' nt p = (fun i → match i with
| Inl () → Inl (NT nt)
| Inm m → (
  if List.mem nt (List.map fst m.rules7) then Inm m
  else (
    let sym = sym_of_parser p in
    let new_rule = (nt, Atom sym) in
    p (Inm (m with rules7 = (new_rule :: m.rules7))))))
| Inr r → (
  let Inr rs = p i in
  Inr (unique rs)))
```

For the left component, *mkntparser'* simply returns a symbol NT nt corresponding to the user supplied label nt . For the middle component, the parser p has a corresponding symbol sym . In terms of the grammar, we should add a new rule $nt \rightarrow sym$. Thus, when passed an argument Inm m we add this new rule before recursively invoking the underlying parser p . The right component is unchanged except that as an optimization we return only unique results.

As well as *mkntparser*, we have an auxiliary function *mktmparser* whose purpose is similar: to introduce concrete names for terminals. This is necessary because the middle component m , as well as accumulating the grammar rules in the field $m.rules7$, also accumulates named terminal parsers in the field $m.tmparsers7$.

7. Example

We can now define an example parser, using the definitions of the previous sections. At this stage, we have no way to construct an oracle automatically, so we will hand code this aspect of the parser. In addition, we have not dealt with the parsing context, so we will not be able to handle all context-free grammars. However, this example will allow us to show how we can extract concrete representations of grammars from parsers.

We will construct a parser for the grammar $E \rightarrow E E E \mid "1"$. We will make use of the raw parser *raw_a1* from Section 4. First, we define our terminal parser:

```
let a1 = mktmparser "1" raw_a1
```

We can now define the parser *parse_E*. For the actions, we will simply count the number of 1s that we parse.

```
let rec parse_E = (fun i → mkntparser' "E" (
  ((parse_E ⊗ parse_E ⊗ parse_E) >> (fun (x, (y, z)) → x + y + z))
  ⊕ (a1 >> (fun _ → 1))))
i)
```

The definition is straightforward. Recall that the sequencing combinator makes use of the oracle to split the input; in order to run our parser on some input, we need to deal with the oracle in some way. At this point, we simply hand code the oracle.

The role of the oracle is to determine, given two symbols $sym1, sym2$, where to cut an input substring $SS(s, i, j)$ into two pieces $SS(s, i, k)$ and $SS(s, k, j)$, so that the first can be parsed as $sym1$ and the second can be parsed as $sym2$.

```
let oracle = (fun (sym1, sym2) → fun (SS(s, i, j)) → ...)
```

For *parse_E* there are two uses of the sequencing combinator: one corresponding to the occurrence of the combinator in the expression $parse_E \otimes parse_E$, and one to the first occurrence of the

combinator in the expression $\text{parse_E} \otimes (\text{parse_E} \otimes \text{parse_E})^7$. There are two nonterminals that can occur as arguments to the sequencing combinator, the nonterminal E , and the nonterminal $(E * E)$ corresponding to the parser $\text{parse_E} \otimes \text{parse_E}$. Note that nonterminal E corresponds to inputs which are non-empty sequences of the character 1. So nonterminal $(E * E)$ corresponds to sequences of 1s of length at least two. We introduce an auxiliary function:

```
let rec upto' i j = (
  if i + 1 < j then (i + 1) :: (upto' (i + 1) j) else [])
```

Note that $\text{upto}' i j = [i + 1; \dots; j - 1]$. Then for this example, the oracle may be coded as follows:

```
let oracle = (fun (sym1, sym2) → fun (SS(s, i, j)) →
  match (sym1, sym2) with
  | (NT "E", NT "(E * E)") → (upto' i (j - 1))
  | (NT "E", NT "E") → (upto' i j))
```

We can define a function to run a parser on a given input, assuming the existence of the oracle:

```
let run_parser3' oracle p s = (
  let i0 = (
    ss4 = (SS(s, 0, String.length s));
    lc4 = empty_context;
    oracle4 = oracle )
  in
  let rs = dest_inr (p (Inr i0)) in
  unique rs)
```

Function $\text{run_parser3}'$ simply evaluates the right component of the parser and returns unique results. We can now run the parser in the OCaml top-level, and OCaml responds with the expected result:

```
let _ = run_parser3' oracle parse_E "1111111"
- : int list = [7]
```

This example illustrates that our parsing combinators are rather close to traditional parsing combinators, with the main difference being the incorporation of the oracle. It is worth noting that the oracle can provide an over-approximation to the possible values for k , without compromising correctness: if the input is split at a value of k for which no possible parses can be found, then the underlying combinator parsers will return an empty list of values. Thus, when defining the oracle for this example, it is safe to use the following:

```
let oracle = fun (sym1, sym2) → fun (SS(s, i, j)) → (upto' i j)
```

Of course, attempting to split the input at positions k for which there are no valid parses will result in additional unnecessary work: this over-approximation will affect performance even if it does not affect correctness.

We can now examine the left and middle components of our example parser.

```
let _ = sym_of_parser parse_E
- : symbol = NT "E"
```

More interesting is the middle component:

```
let m = grammar_of_parser parse_E
val m: mid =
<rules7 =
  [ ("(E * E)", Seq (NT "E", NT "E"));
    "(E * (E * E))", Seq (NT "E", NT "(E * E))");
    "((E * (E * E)) + 1)", Alt (NT "(E * (E * E))", TM "1"));
    "E", Atom (NT "((E * (E * E)) + 1)"));
  imparsers7 = [ ("1", <fun >)]
```

The result is a record m . The $m.\text{rules7}$ field contains a concrete representation of the grammar, with nonterminals corresponding to every use of the sequencing and alternative combinators. In addition, the $m.\text{imparsers7}$ field represents a finite map from terminals to the corresponding raw parsers. In this example, there is only one entry for the terminal "1".

In this section we have worked through the definition of a simple parser, and seen how the machinery introduced in previous sections allows us to extract a concrete representation of the grammar from code such as parse_E . With a concrete representation of the grammar, we can then use a method such as Earley parsing to perform a parse of the input, determine the information necessary to construct an oracle, and then finally use the oracle to seed the action phase of the parse.

8. Earley parsing and construction of the oracle

In this section we describe how we construct an oracle from the results of an Earley parse. For an explanation of Earley parsing itself we refer the reader to the original paper [5].

Earley parsing involves Earley items, which are tuples of the form (X, α, β, i, j) , typically written $(X \rightarrow \alpha.\beta, i, j)$. These items are relative to some input string s and grammar Γ . Such an item can only arise if there is a rule $X \rightarrow \alpha \beta \in \Gamma$. The interpretation of such an item is that $\Gamma \vdash \alpha \rightarrow^* s_{i,j}$.

Earley parsing is initialized with a single Earley item $(S \rightarrow \beta, 0, 0)$, where S is the start symbol. When Earley parsing finishes, it returns a finite set of items, which include all items which correspond to parse trees for the input s . A complete set of Earley items represents a *compact representation* of all possible parse trees.

It is possible to construct an Earley parser that returns Earley productions rather than items. An Earley production is a pair consisting of an item and an additional integer l , written $(X \rightarrow \alpha.\beta, i, j, l)$. For productions, the β component is non-empty, that is, β is of the form $Y\beta'$ for some symbol Y . A production $(X \rightarrow \alpha.Y\beta', i, j, l)$ should be interpreted as follows: $(X \rightarrow \alpha.Y\beta', i, j)$ is an Earley item i.e. $\Gamma \vdash \alpha \rightarrow^* s_{i,j}$, and $\Gamma \vdash Y \rightarrow^* s_{j,l}$.

In cases where α is non-empty, a production has the form $(X \rightarrow \alpha'Z.Y\beta', i, j, l)$. If we work with grammars where the right-hand-side has at most two symbols, such a production has the form $(X \rightarrow Z.Y, i, j, l)$. These productions provide exactly the information required by the oracle: if we wish to parse the input between positions i and l , for the symbols Z and Y , we should cut the input at position j (note that the cut position is now the j component of the Earley item). We introduce the following types to represent Earley items and productions:

```
type item = <
  nt2 : nonterm;
  a2 : symbol list;
  b2 : symbol list;
  i2 : int;
  j2 : int >
```

```
type production = item × int
```

In the P3 distribution, we provide a purely functional implementation of an Earley parser. The interface to this function is via the following function:

```
let earley_prods_of_parser p s = ...
```

This is a function of type $\alpha \text{ parser3} \rightarrow \text{string} \rightarrow \text{production list}$. Given a parser and an input string, it returns a list of productions. We can process these productions to produce an oracle.

⁷ Recall that the sequencing combinator associates to the right.

```

let oracle_of_prods ps = (fun (sym1, sym2) → fun (SS(s, i', j')) →
  let fl (itm, l) = (itm.a2 = [sym1])
    && (itm.b2 = [sym2])
    && (i', j') = (itm.i2, l)
  in
  let ps = List.filter fl ps in
  List.map (fun (itm, _) → itm.j2) ps)

```

Applying *oracle_of_prods* to a list of productions *ps* gives an oracle, that is, a function of type $(\text{symbol} \times \text{symbol}) \rightarrow \text{substring} \rightarrow \text{int list}$. The function simply picks out those productions $(X \rightarrow \alpha.\beta, i, j, l)$ where α is the single symbol *sym1*, β is the single symbol *sym2*, and the span (i, l) corresponds to the span (i', j') of the substring that we want to parse. The cut indexes are simply those *j* for such productions⁸.

9. Example, with Earley parsing

We continue the example from Section 7. Deriving the productions for a given input is straightforward:

```
let ps = earley_prods_of_parser parse_E "1111111"
```

We can then construct an oracle based on these productions:

```
let oracle = oracle_of_prods ps
```

We can query the oracle, for example, to find out where to split the input if we wish to parse a sequence of two symbols:

```
let _ = oracle (NT "E", NT "(E*E)") (SS("1111111", 0, 7))
- : int list = [1; 3; 5]
```

The resulting list $[1; 3; 5]$ reveals that the sequence of two non-terminals *E* (*E*E*) can be used to parse an input "1111111" by splitting the input at positions 1, 3 and 5.

In Section 7 we hand coded the oracle. We can now improve on this by automatically constructing the oracle from the parser itself.

```

let run_parser3 p s = (
  let ps = earley_prods_of_parser p s in
  let oracle = oracle_of_prods ps in
  run_parser3' oracle p s)

```

We can then run our parser in the OCaml top-level and examine the result:

```

let _ = run_parser3 parse_E "1111111"
- : int list = [7]

```

10. Context

We cannot yet handle all context-free grammars, at least during the action phase. For this, we need the notion of parsing context. In this section we recall the essential definitions. We also show how parsing context may be smoothly combined with the machinery developed in previous sections. For reasons of space, we give an informal presentation, covering the main ideas, but omitting formal definitions and theorems which can be found in [15].

Consider the grammar $E \rightarrow E E E \mid "1" \mid \epsilon$. This grammar is highly ambiguous. It also gives rise to infinitely many parse trees. In Fig. 3 we show a parse tree formed from this grammar (the leaves are decorated with strings rather than substrings). Note that *pt* and *pt'* are rooted at the same nonterminal, correspond to the same string (a string consisting of a single 1), and *pt'* is properly contained in *pt*. If such a situation can occur then the grammar gives rise to an infinite number of parse trees. Moreover, *this is the only way an infinite number of parse trees can arise*. We call a parse

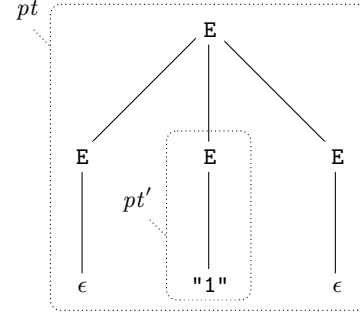


Figure 3.

tree bad if it contains a subtree such as *pt*. If we rule out bad trees, we can still find a good tree for any parse-able input: the operation of replacing subtrees such as *pt* by subtrees *pt'* that are contained in *pt* defines a reduction process on parse trees, which preserves the parsed input. Thus, for a given grammar we have identified a class of parse trees (the good parse trees) that is complete (any string that can be parsed, can be parsed to give a good parse tree) and moreover is *finite*.

We wish to restrict parsing to only the good parse trees. To this end, at a given node the combinator parser records the current nonterminal and input string in a *parsing context*. If the parser encounters the same nonterminal and input string while parsing a subtree, the parse should be abandoned. The context consists of a finite set (represented by a list) of pairs, where each pair consists of a nonterminal, and the input substring.

For example, at the root of *pt*, the context is initially empty. The parser adds the entry $(\text{"E"}, \text{SS}(\text{"1"}, 0, 1))$ corresponding to the root of *pt* before parsing the subtrees. When the parser reaches *pt'*, it constructs the entry $(\text{"E"}, \text{SS}(\text{"1"}, 0, 1))$ corresponding to the root of *pt'* and checks whether it is already contained in the context. In this case, the entry *is* contained in the context, and the parse is abandoned.

The implementation of contexts (finite sets) as lists is straightforward. We need a function to update the context at a node, and a function to check whether the item corresponding to a node is already contained in the context.

```
let update_context (LC(lc)) (nt, SS(s, l, h)) = (
  LC((nt, SS(s, l, h)) :: lc))
```

```
let context_contains (LC(lc)) (nt, SS(s, l, h)) = (
  List.exists (fun x → x = (nt, SS(s, l, h))) lc)
```

We can then lift the function *update_context* to the input type:

```

let update_lc4 nt p = (fun i → match i with
  | lnr i → p (lnr (
    i with lc4 = (update_context i.lc4 (nt, i.ss4))))
  | _ → p i)

```

This function takes a string *nt* such as "E" representing the nonterminal at the current node, and a parser *p* representing the parser for that nonterminal, and updates the context before calling the underlying parser on the input.

When executing a parser for nonterminal *nt* on input *i*, we first need to check whether the entry $(nt, i.ss4)$ already occurs in the context. If so, we should abandon the parse. Otherwise, we should update the context and continue.

⁸The definition of *oracle_of_prods* given here is solely for expository purposes; a more efficient implementation is provided in the online resources.

```

let check_and_upd_lc4 p = (fun i → match i with
| Inr i → (
  let (f, g, h) = unsum3 p in
  let NT nt = sym_of_parser p in
  let should_trim = context_contains i.lc4 (nt, i.ss4) in
  if should_trim then
    Inr []
  else
    update_lc4 nt p (Inr i))
| _ → p i)

```

The function *check_and_upd_lc4 p* affects only the right component. It first determines the nonterminal *nt* corresponding to *p*. It then checks whether the entry *(nt, i.ss4)* is already contained in the context. If so, it abandons the parse and returns the empty list of results. Otherwise it updates the context and calls the underlying parser *p*. Finally, we construct the function *mkntparser*:

```

let mkntparser nt p = (
  check_and_upd_lc4 (mkntparser' nt p))

```

With these definitions, we can now handle *all* context-free grammars.

We make two further observations that are relevant to the discussion in Section 12 of the memoization of functions (such as parsers) which take contexts as arguments. The first is that contexts are finite sets, so that we might consider using *sorted* lists to represent contexts. The second observation is that many contexts are equivalent in the way they affect a parse. Suppose the entry for the current node is of the form *(nt, SS(s, i, j))*. All entries in the context will be of the form *(nt', SS(s, i', j'))*, where $i' \leq i, j \leq j'$. If in fact $i' < i$ (or $j < j'$) then that entry in the context will not cause the parse to be abandoned at the current node. Nor will it cause the parse to be abandoned for any descendants of the current node. Such entries are therefore irrelevant. Contexts which are identical but for the presence of irrelevant entries are equivalent in the way they affect a parse. Given a substring *ss*, we can normalize a given context to remove irrelevant entries:

```

let normalize_context (LC(lc)) ss = (
  LC(List.filter (fun (nt, ss') → ss' = ss) lc))

```

In Section 12, memoization performs best when contexts are *normalized*, and represented using *sorted* lists.

11. Example, with context

Consider the grammar $E \rightarrow E E E \mid "1" \mid \epsilon$. We can define a parser for this grammar as follows. As before, the actions simply count the number of 1 characters consumed.

```

let rec parse_E = (fun i → mkntparser "E" (
  ((parse_E ⊗ parse_E ⊗ parse_E) >> (fun (x, (y, z)) → x + y + z))
  ⊕ (a1 >> (fun _ → 1))
  ⊕ (eps >> (fun _ → 0))))
i)

```

Compared to previous examples, the difference is that we can now parse the empty string ϵ as the nonterminal "E". This in turn gives rise to an infinite number of parse trees. However, our parsers can now handle such grammars easily.

```

let _ = run_parser3 parse_E "1111111"
- : int list = [7]

```

At this point, we have developed parser combinators that can handle all context-free grammars. However, parsing is still slow because, during the action phase, a parser such as *parse_E* may be called many times on the same substring. To make our parsing efficient we need to use memoization to avoid repeated work.

12. Memoization

Memoization of a function *f* involves maintaining a partial map, known as a lookup table, from arguments *i* to values *f i*. Instead of computing *f* on a argument *i*, memoization first checks whether the value is already stored in the table. If so, the value is returned without further computation involving *f*. Otherwise, the value of *f i* is computed, the table is updated, and the value is returned. Memoization is only valid if *f* behaves as a pure function. A direct implementation using OCaml hashtables is as follows:

```

let memo tbl f i = (
  if (Hashtbl.mem tbl i) then (Hashtbl.find tbl i)
  else (
    let v = f i in
    let _ = Hashtbl.add tbl i v in
    v))

```

Here, arguments are uninterpreted, however, it is often the case that several arguments $i_1, i_2, \dots$ should be considered equivalent because the values $f i_1, f i_2, \dots$ are the same. If we have stored the result of evaluating $f i_1$ in the memo table, we would like to reuse this value to avoid computing $f i_2$. To handle this case, we extend the *memo* function with an auxiliary function *key_of_input* that computes a key *k* from an input *i*. It is expected that $i_1, i_2, \dots$ will all map to the same key, and arguments i, i' should map to the same key only if $f i = f i'$. The index into the memoization table is now the key, rather than the argument itself.

```

let memo tbl key_of_input f i = (
  let k = key_of_input i in
  match k with
  | None → (f i)
  | Some k → (
    if (Hashtbl.mem tbl k) then (Hashtbl.find tbl k)
    else
      let v = f i in
      let _ = Hashtbl.add tbl k v in
      v))

```

The function *key_of_input* may return *None*, which indicates that the value for the given argument should not be memoized. For a parser *p*, we wish to memoize only the right component.

```

let key_of_input i = (match i with
| Inr i → (
  let lc4 = normalize_context i.lc4 i.ss4 in
  let k = (lc4, i.ss4) in
  Some k)
| _ → None)

```

Note that the right component of a parser accepts arguments *Inr i* which, in addition to the substring component *i.ss4* also include a context *i.lc4* and an oracle *i.oracle4*. The oracle does not change during the action phase, and so need not be considered when memoizing. So the input to a parser is effectively a pair of a context and a substring. As observed in Section 10, memoization performs best when the context is normalized relative to the input string at the current node, so we use the function *normalize_context* to normalize *i.lc4* when constructing the key⁹. It is now straightforward to memoize a parser:

⁹Memoization using keys provided by *key_of_input* implicitly requires computing a hash code for the substring component *i.ss4*. The input string has the potential to be very large. It is well known that repeated computation of hash codes can change the time complexity of the underlying algorithm, perhaps resulting in asymptotically *worse* performance. However, when working with a single input string, the relevant parts of a substring *SS(s, i, j)* are the indices *i, j*. P3 differs from mini-P3 in that it discards the string component *s* when computing keys for memoization. A more prin-

```
let memo_p3 tbl p i = (memo tbl key_of_input p i)
```

13. Example, with Earley parsing, context and memoization

An efficient parser for the grammar $E \rightarrow E E E \mid "1" \mid \epsilon$ can now be presented in its entirety.

```
let tbl = Hashtbl.create 0
```

```
let rec parse_E = (fun i -> memo_p3 tbl (mktparser "E" (
  ((parse_E ⊗ parse_E ⊗ parse_E) >> (fun (x, (y, z)) -> x + y + z))
  ⊕ (a1 >> (fun _ -> 1)))
  ⊕ (eps >> (fun _ -> 0))))
  i)
```

This mini-P3 parser can parse an input consisting of a sequence of 1 characters of length 100, apply the actions, and return the single result in 9s on current hardware (using a single core). The equivalent parser using the full P3 library returns a single result in the reduced time of 6s, since it does not repeatedly compute the hash code of the input string. By way of comparison, the Haskell Happy GLR parser takes over 40 minutes just to compute the compact representation of the parse trees (equivalent to the computation of the Earley productions).

14. Experiments and performance

In this section we discuss performance, mainly by comparing our approach to the popular Haskell Happy parser generator [1]. Happy is a mature GLR parser capable of parsing all context-free grammars. We assess the performance of P3 and Happy across 5 different grammars. On some of these grammars, P3 outperforms Happy, and on some Happy outperforms P3. The core Earley parser used by P3 is essentially unoptimized, and there are clear opportunities to improve the performance, so these initial results are extremely encouraging.

Why Happy? We should compare P3 against a parser that can handle all context-free grammars: On restricted classes of grammar, we expect that P3 has good asymptotic performance, but absolute performance will not compare favourably with specialized parsing techniques. For example, P3 should probably not be used to parse XML if absolute performance is the priority. We carried out preliminary experiments with general parsers such as AC-CENT¹⁰, Elkhound¹¹ and SPARK¹², but encountered problems that were seemingly hard to resolve. For example, the author of SPARK confirmed that SPARK cannot directly handle grammars such as $E \rightarrow E E E \mid "1" \mid \epsilon$. The underlying reason appears to be that SPARK does not make use of a compact representation of parse trees, but works instead with abstract syntax trees, which is problematic in this case because a single input can give rise to a possibly infinite number of parse trees. On the other hand, it was relatively straightforward to code up example grammars in Happy, and extract the results using a compact representation. We believe Happy represents a demanding target for comparison because it is mature, well-tested and extensively optimized code. For example, the authors of the Parsec library take Happy performance to be almost the definition of efficiency¹³.

cipled approach would use the techniques of boxing and hash-consing, see e.g. [2]. We plan to incorporate such a change in the next release.

¹⁰ <http://accent.compilertools.net/>

¹¹ <http://scottmcpeak.com/elkhound/>

¹² <http://pages.cpsc.ucalgary.ca/~aycock/spark/>

¹³ "[Our real-world requirements on the combinators]...they had to be efficient (ie. competitive in speed with happy and without space leaks)" [12]

What to measure? We measure the time taken for each of the three phases separately. First we compare the time to compute a compact representation of all parses. This involves comparing our core implementation of Earley’s algorithm with the core GLR implementation in Happy. Second, we examine the overhead of constructing the oracle. Third, we examine the overhead of applying parsing actions. As a very rough guide, we expect the Earley parsing phase to be $O(n^3)$. The construction of the oracle involves iterating over the list of productions, which is $O(n^3)$ in length, so we might expect that this phase should also take time $O(n^3)$. The time taken to apply the actions depends on the actions themselves, but we can analyse particular actions in an ad hoc manner to check that the observed times for this phase are reasonable.

Grammars and inputs We selected 5 grammars as representative examples of general context-free grammars. These are:

Identifier	Grammar
aho_s	$S \rightarrow "x" S S \mid \epsilon$
aho_sml	$S \rightarrow S S "x" \mid \epsilon$
brackets	$E \rightarrow E E \mid "(" E ")" \mid \epsilon$
E_EEE	$E \rightarrow E E E \mid "1" \mid \epsilon$
S_xSx	$S \rightarrow "1" S "1" \mid "1"$

The grammars aho_s and aho_sml are taken from a well-known book on parsing [3]. They were used to assess parser performance in related work [8]. The grammar brackets is a simple grammar for well-bracketed expressions. The grammar E_EEE is the example grammar we have used throughout the paper. It has no “left-right” or “right-left” bias, which is not the case for aho_s and aho_sml. The final grammar S_xSx is an example of a non-ambiguous grammar that cannot be handled using Packrat parsing, taken from [6].

We used binarized versions of these grammars when measuring the performance of our Earley parser, because the P3 library feeds only binarized grammars to the Earley parser. We tried to check whether binarized versions of the grammars improved the performance of Happy, but at least with a binarized version of the grammar E_EEE, Happy appeared to hang on non-empty input strings.

For inputs, we simply used strings consisting of the characters x or 1, or well-bracketed expressions, of varying lengths. For S_xSx all inputs were of odd length.

Results: computation of compact representation For the grammars aho_s and E_EEE, our Earley parser clearly outperformed Happy. For example, here are the results for aho_s¹⁴:

Size	Happy parse time	Earley parse time
20	0.13	0.02
40	2.91	0.02
60	26.55	0.08
80	135.10	0.18
100	475.10	0.34

For the grammars aho_sml and S_xSx, Happy outperformed Earley, although the results are quite close (within a small factor). For example, here are the results for S_xSx:

¹⁴ All times in this section are measured in seconds. All sizes are measured in characters.

Size	Happy parse time	Earley parse time
301	0.14	0.47
401	0.29	0.89
501	0.53	1.5
1001	3.63	8.21
2001	27.22	66.41

Finally the grammar brackets caused Happy to appear to loop when parsing input, possibly due to a bug in Happy¹⁵.

In conclusion, on two of the grammars, Earley is much more performant than Happy. On two of the grammars, Happy is more performant than Earley, but the results are quite close. On the remaining grammar, Happy appeared to loop when attempting to parse inputs.

We now emphasize that the contribution of this work is *not* an efficient implementation of Earley parsing. Any general approach to parsing, such as GLR parsing, could be used to construct the oracle. We implemented our own version of Earley parsing because we could not find a general parser written in OCaml that we could easily adapt to our purposes. In fact, our implementation of Earley has been designed primarily so that it can be verified in a theorem prover. Our core implementation is 60 lines of pure, unoptimized OCaml code. As such, we should expect that our implementation will not have good absolute performance. The fact that it does is rather unexpected. It is a *strength* of our approach that the underlying parsing technique used to construct the oracle can be switched (in order to improve performance) without altering the parsing library interface, which is based on parser combinators.

In addition to absolute performance, we can also check whether our Earley parser has the expected time complexity. We have observed that our Earley implementation performs worse than $O(n^3)$ for grammars `aho_sml` and `E.EEE` on large inputs. We believe this is because our purely functional code makes extensive use of functional implementations of sets and maps. Replacing these functional implementations with lookups into mutable arrays may deliver the expected performance, but we leave this to future work. Our parser is still competitive with Happy in absolute terms because Happy appears to suffer from even worse asymptotic performance.

Results: constructing the oracle We now examine the overhead of constructing the oracle. How long should we expect the construction of the oracle to take? The oracle is constructed by iterating over the Earley productions. Since there are $O(n^3)$ productions, this stage should be $O(n^3)$ at best. In practice, our implementation processes each production by inserting an element into a functional map, so that the time to process each production is not constant. Thus, we observe slightly worse than $O(n^3)$ performance. For example, for the grammar `E.EEE`, the times for the Earley phase, and the times to construct the oracle, are:

Size	Earley parse time	Oracle construction time
20	0.008	0.004
40	0.052	0.040
60	0.148	0.148
80	0.348	0.392
100	0.724	0.864

Using a naive estimation technique puts the oracle construction time at $O(n^{3.5})$. We believe that an implementation using mutable arrays would achieve the expected $O(n^3)$ bound.

Results: applying parsing actions We now examine the overhead of applying parsing actions. Our approach restricts to good parse

trees, which are finite in number. Parsers such as Happy do not restrict to good parse trees, and so attempting to construct parse trees, or apply actions to, parsing results for a grammar such as `E → E E E | "1" | ε` will result in non-termination. Thus, it is not possible to compare the performance of P3 and Happy in this area, but we can look at the behaviour of P3 itself.

How long should we expect the action phase to take? Consider the grammar `S → "x" S S | ε`, where the actions count the number of characters parsed. Without memoization we expect the action phase to take an exponential amount of time. With memoization we can argue as follows. Suppose the time to apply the actions is dominated by the non-memoized recursive calls, so that we can ignore the time taken for memoized calls. There are $O(n^2)$ non-memoized calls to parse an `S` (corresponding to different spans (i, j) of the input string). For each call, the input must be split in $O(n)$ places, and the single result from each subparse combined. Thus, each call takes $O(n)$ time, giving an overall $O(n^3)$ execution time for the action phase. In practice, the time taken to look up a precomputed value in the memoization table cannot be ignored, thus we observe slightly worse than $O(n^3)$ performance. In the following, we include times for all phases to give an idea of the relative costs. Using a naive estimation technique puts the action phase at $O(n^{3.1})$.

Size	Earley phase	Oracle phase	Action phase
100	0.348	0.304	0.204
200	2.80	2.85	1.47
300	10.1	10.8	5.06
400	25.6	28.3	12.7
500	56.1	67.5	25.4

For the grammars `aho_sml` and `E.EEE` one can reason similarly. The grammar brackets might be expected to have similar performance $O(n^3)$ when applying the actions. If the input consists of alternating left and right brackets, this is the case. Our example inputs consist of relatively few blocks of deeply nested brackets, so that most recursive calls execute in constant time rather than $O(n)$, and overall the observed performance is closer to $O(n^2)$ for the input sizes we have tested (upto 120,000 characters). Finally, consider the following code for the grammar `S.xSx`:

```
let rec parse_SxSx = (fun i → memo.p3 tbl (mkntparser "S" (
  ((a1 ⊗ parse_SxSx ⊗ a1) >> (fun (_, (x, -)) → 2 + x))
  ⊕ (a1 >> (fun _ → 1))))
i)
```

For an input of length $n + 1$ there should be $n/2$ recursive calls when applying the actions, each of which takes a constant time to execute, giving expected $O(n)$ cost for applying the actions. In practice, the time to apply the actions is negligible compared to the other two phases. For example, for input length 2001, the Earley parser takes 66 seconds, constructing the oracle takes 58 seconds, and applying the actions takes 0.012 seconds.

Conclusions The Earley parser appears competitive with Happy: for some grammars it is slightly worse, and for some better. We are pleased with these results: our Earley parser is unoptimized, and there are clear areas where the performance can be improved, both asymptotically (using mutable arrays rather than functional data-structures) and in absolute terms. Moreover, the performance of our Earley parser is not critical: our approach can be adapted to use any general parsing implementation when constructing the oracle, so we can take advantage of faster, optimized parsing implementations when they become available.

Constructing the oracle currently involves processing each of the $O(n^3)$ productions from the Earley parsing stage. A more

¹⁵ Reported to the authors of Happy on 2013-06-24.

intelligent approach would be to process only those productions that contribute to a valid parse. For example, for the grammar $S \rightarrow xSx$ there are only $O(n)$ such items. This optimization should reduce the oracle construction time significantly for some grammars.

Finally, the observed cost of applying the actions for our chosen grammars agrees with a basic complexity analysis, and we do not see much scope for reducing this time further.

15. Related work

Research on parsing has been carried out over many decades by many researchers. We cannot hope to survey all of this existing work, and so we here restrict ourselves to consideration of only the most directly related work. The first parsing techniques that can handle arbitrary context-free grammars are based on dynamic programming. Examples include CYK parsing [11] and Earley parsing [5]. The popular GLR parsing approach was introduced in [18]. Combinator parsing and related techniques are probably folklore. An early approach with some similarities is [14]. Versions that are clearly related to the approach taken in this paper were popularized in [10].

The extension of combinator parsing to handle all context-free grammars using a parsing context, as in this paper, appears in [15]. As described in that paper, the use of a parsing context is related to a long line of work that uses the length of the input to force termination [7–9].

Our work is motivated by the desire to provide a combinator parsing interface with performance competitive with $O(n^3)$ general algorithms such as Earley parsing. In [16] the authors “develop the fully general GLL parsing technique which is recursive descent-like, and has the property that the parse follows closely the structure of the grammar rules”. The desire is to improve on the shortcomings of GLR: “Nobody could accuse a GLR implementation of a parser for, say, C++, of being easy to read, and by extension easy to debug.” This work is very similar in its aims to ours. Prototype hand-coded implementations of recognizers for several grammars, based on the GLL algorithm, are described in [16]. These do not provide a combinator parsing interface. An implementation of GLL in Scala that provides the desired combinator parsing interface is described in [17], but the author admits “at the moment, performance is basically non-existent.” However, we believe that the GLL algorithm represents the main competition to our approach and we eagerly await future efficient implementations which provide a combinator parsing interface.

16. Conclusion and future work

We presented an approach to parsing that provides a flexible interface based on parsing combinators, together with the performance of general approaches such as Earley parsing. One of the main benefits of our approach is its simplicity. The code itself is straightforward, and short, which should make it easy to port to other functional languages. The code, and our presentation in this paper, is modular — it smoothly combines different aspects such as grammar extraction, parsing with context, and memoization — which gives many benefits, such as aiding comprehension. For the future, we plan to implement several improvements to performance, some of which we noted in Section 14. Another area that we plan to improve is feedback to the user, which is important e.g. when an input fails to parse at all, or when an input parses in several different ways when it is expected to parse unambiguously. In addition, we hope to verify our approach, following previous work such as [15]. As a first step, we plan to verify functional correctness of our implementation of Earley parsing.

References

- [1] Happy, a parser generator for Haskell. <http://www.haskell.org/happy/>.
- [2] U. A. Acar, G. E. Blelloch, and R. Harper. Selective memoization. *SIGPLAN Not.*, 38(1):14–25, Jan. 2003. ISSN 0362-1340. doi: 10.1145/640128.604133. URL <http://doi.acm.org/10.1145/640128.604133>.
- [3] A. V. Aho and J. D. Ullman. *The theory of parsing, translation, and compiling*. Prentice-Hall, Inc., 1972.
- [4] A. Barthwal and M. Norrish. A mechanisation of some context-free language theory in HOL4. *Journal of Computer and System Sciences*, 2013.
- [5] J. Earley. An efficient context-free parsing algorithm. *Commun. ACM*, 13(2):94–102, 1970. ISSN 0001-0782. doi: <http://doi.acm.org/10.1145/362007.362035>.
- [6] B. Ford. Packrat parsing: simple, powerful, lazy, linear time, functional pearl. In *ICFP 02: Proceedings of the seventh ACM SIGPLAN international conference on Functional programming*, volume 37/9, pages 36–47, New York, NY, USA, 2002. ACM. doi: 10.1145/583852.581483. URL <http://pdos.csail.mit.edu/~baford/packrat/icfp02/packrat-icfp02.pdf>.
- [7] R. A. Frost, R. Hafiz, and P. C. Callaghan. Modular and efficient top-down parsing for ambiguous left-recursive grammars. In *IWPT '07: Proceedings of the 10th International Conference on Parsing Technologies*, pages 109–120, Morristown, NJ, USA, 2007. Association for Computational Linguistics. ISBN 978-1-932432-90-9.
- [8] R. A. Frost, R. Hafiz, and P. Callaghan. Parser combinators for ambiguous left-recursive grammars. In P. Hudak and D. S. Warren, editors, *PADL*, volume 4902 of *Lecture Notes in Computer Science*, pages 167–181. Springer, 2008. ISBN 978-3-540-77441-9.
- [9] R. Hafiz and R. A. Frost. Lazy combinators for executable specifications of general attribute grammars. In M. Carro and R. Peña, editors, *PADL*, volume 5937 of *Lecture Notes in Computer Science*, pages 167–182. Springer, 2010. ISBN 978-3-642-11502-8.
- [10] G. Hutton. Higher-order functions for parsing. *J. Funct. Program.*, 2(3):323–343, 1992.
- [11] T. Kasami. An efficient recognition and syntax analysis algorithm for context-free languages. Technical Report AFCRL-65-758, Air Force Cambridge Research Laboratory, Bedford, Massachusetts, 1965.
- [12] D. Leijen and E. Meijer. Parsec: A practical parser library. *Electronic Notes in Theoretical Computer Science*, 41(1):1–20, 2001.
- [13] J. M. Leo. A general context-free parsing algorithm running in linear time on every LR(k) grammar without using lookahead. *Theoretical Computer Science*, 82(1):165 – 176, 1991. ISSN 0304-3975. doi: [http://dx.doi.org/10.1016/0304-3975\(91\)90180-A](http://dx.doi.org/10.1016/0304-3975(91)90180-A). URL <http://www.sciencedirect.com/science/article/pii/030439759190180A>.
- [14] V. R. Pratt. Top down operator precedence. In *Proceedings ACM Symposium on Principles Prog. Languages*, 1973.
- [15] T. Ridge. Simple, functional, sound and complete parsing for all context-free grammars. In *Certified Programs and Proofs*, pages 103–118. Springer, 2011.
- [16] E. Scott and A. Johnstone. GLL parsing. *Electronic Notes in Theoretical Computer Science*, 253(7):177–189, 2010.
- [17] D. Spiewak. Generalized parser combinators. 2010. URL <http://www.cs.uwm.edu/~dspiewak/papers/generalized-parser-combinators.pdf>.
- [18] M. Tomita. LR parsers for natural languages. In *Proceedings of the 10th international conference on Computational linguistics*, pages 354–357. Association for Computational Linguistics, 1984.